

CSE_2323

*Multiplexer
&
Demultiplexer*

Digital Logic Design

Course code: CSE-2323

Credit Hour: 3

Md. Mujibur Rahman Maruf

Assistant Lecturer,

Dept. of CSE, IIUC

Mujiburmaruf.cuet17@gmail.com

International Islamic University Chittagong(IIUC)



Mux (Multiplexer)

Multiplexer (MUX)

A **Multiplexer (MUX)** is a combinational circuit that selects one of many input signals and forwards the selected input to a single output line. It's often called a **data selector**.

➤ Multiplexer contains the followings:

- ❖ data inputs
- ❖ selection inputs
- ❖ a single output

➤ Selection input determines the input that should be connected to the output.

➤ The multiplexer acts like an electronic switch that selects one from different.

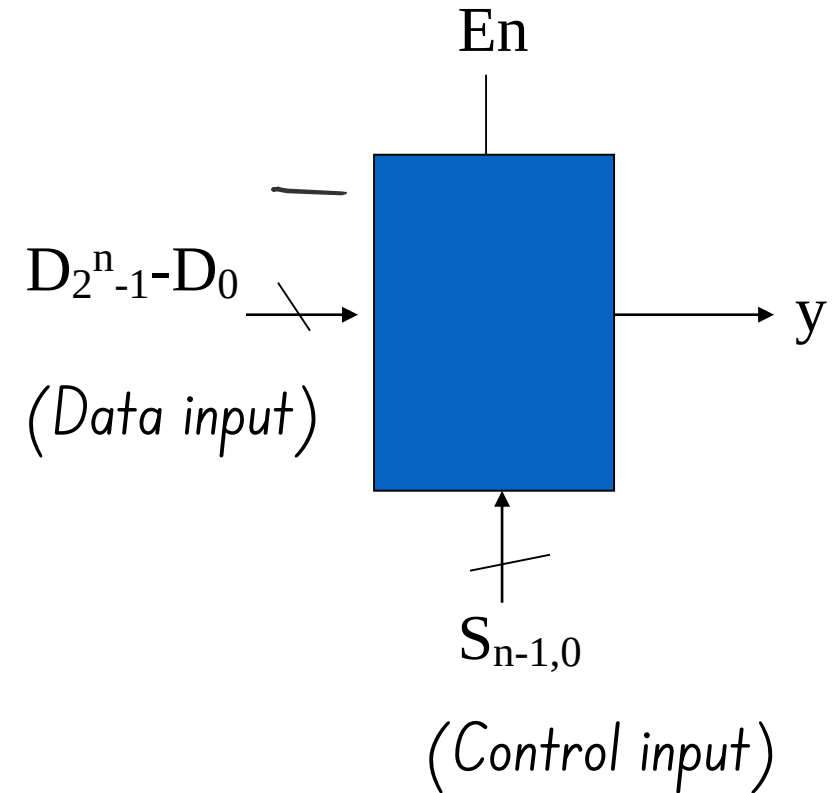
Demultiplexer (DEMUX)

A **Demultiplexer (DEMUX)** is a combinational circuit that takes **one input** and routes it to **one of many outputs**, based on select line values. It works as the **opposite of a multiplexer**.

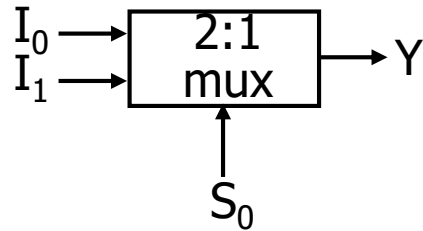
Mux (Multiplexer)

Basic concept

- 2^n data inputs; n control inputs ("selects"); 1 output
- Connects one of 2^n inputs to the output
- "Selects" decide which input connects to output
- Two alternative truth-tables: *Functional* and *Logical*

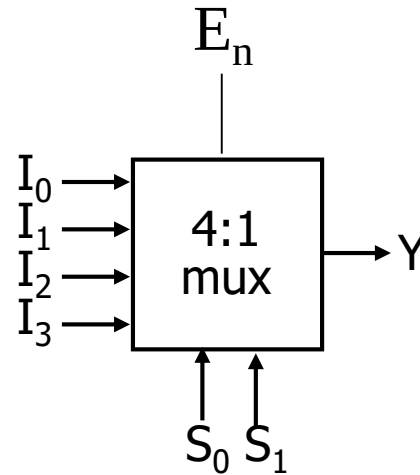


Types of Multiplexers



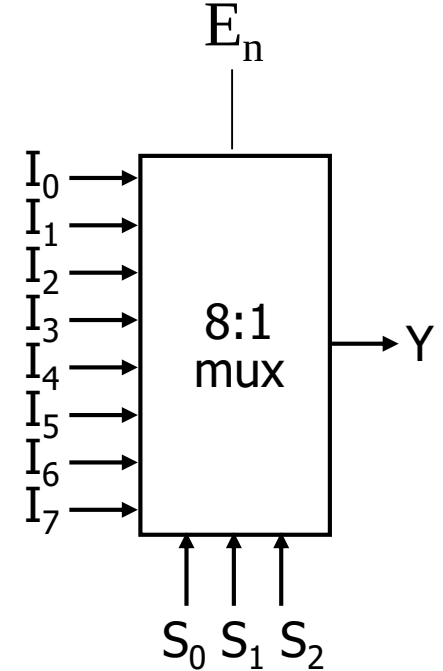
1. 2-to-1 Multiplexer

- Inputs: 2 (I_0, I_1)
- Select Lines: 1 (S_0)
- Output: 1 (Y)
- Function: Selects one of the two inputs based on the value of S_0 .



2. 4-to-1 Multiplexer

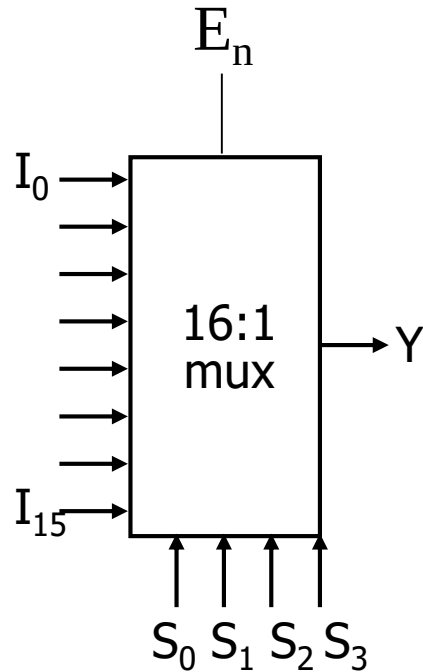
- Inputs: 4 (I_0 to I_3)
- Select Lines: 2 (S_1, S_0)
- Output: 1
- Function: Selects one of four inputs.



3. 8-to-1 Multiplexer

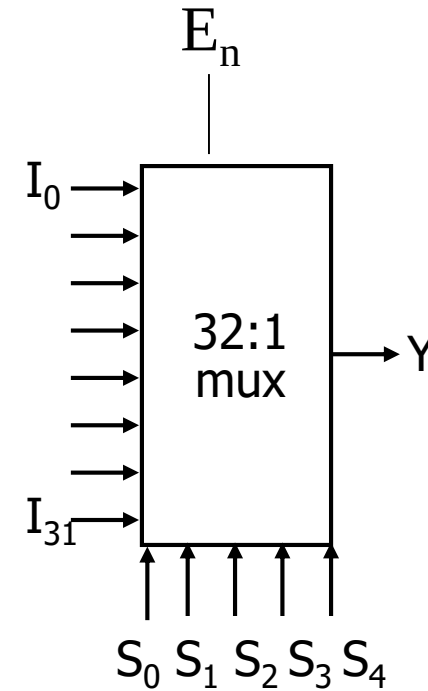
- Inputs: 8 (I_0 to I_7)
- Select Lines: 3 (S_2, S_1, S_0)
- Output: 1

Types of Multiplexers



4. 16-to-1 Multiplexer

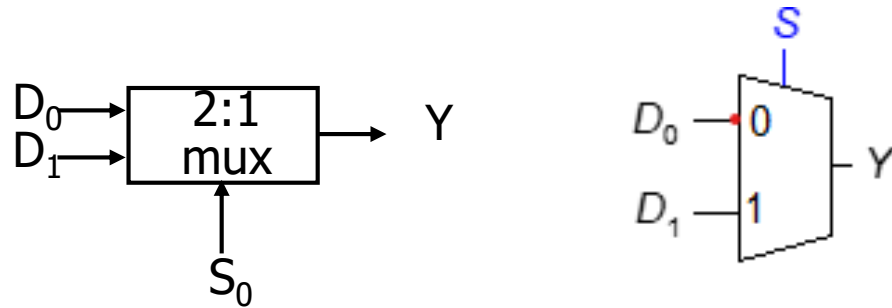
- **Inputs:** 16 (I_0 to I_{15})
- **Select Lines:** 4 (S_3 to S_0)
- **Output:** 1
- **Function:** Selects one of sixteen inputs



5. 32-to-1 Multiplexer

- **Inputs:** 32 (I_0 to I_{31})
- **Select Lines:** 5 (S_4 to S_0)
- **Output:** 1
- **Function:** Selects one of 32 inputs

2x1 Multiplexer



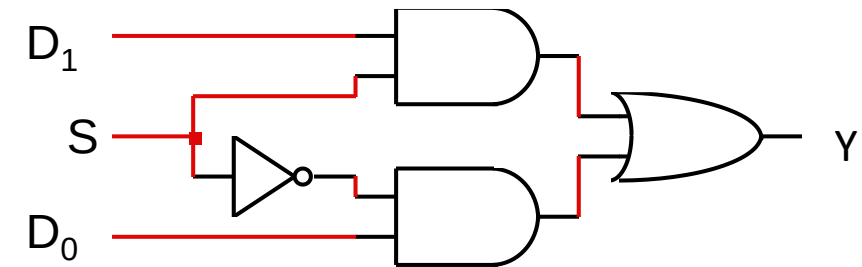
Functional truth table

S	Y
0	D ₀
1	D ₁

The 2x1 mux has two input lines, one output line, and a single selection line. The output of the 2x1 Mux will depend on the selection line S_0 ,

- When S is 0 (low), the I_0 is selected
- when S_0 is 1 (High), I_1 is selected

$$Y = SD_1 + S'D_0$$



4x1 Multiplexer

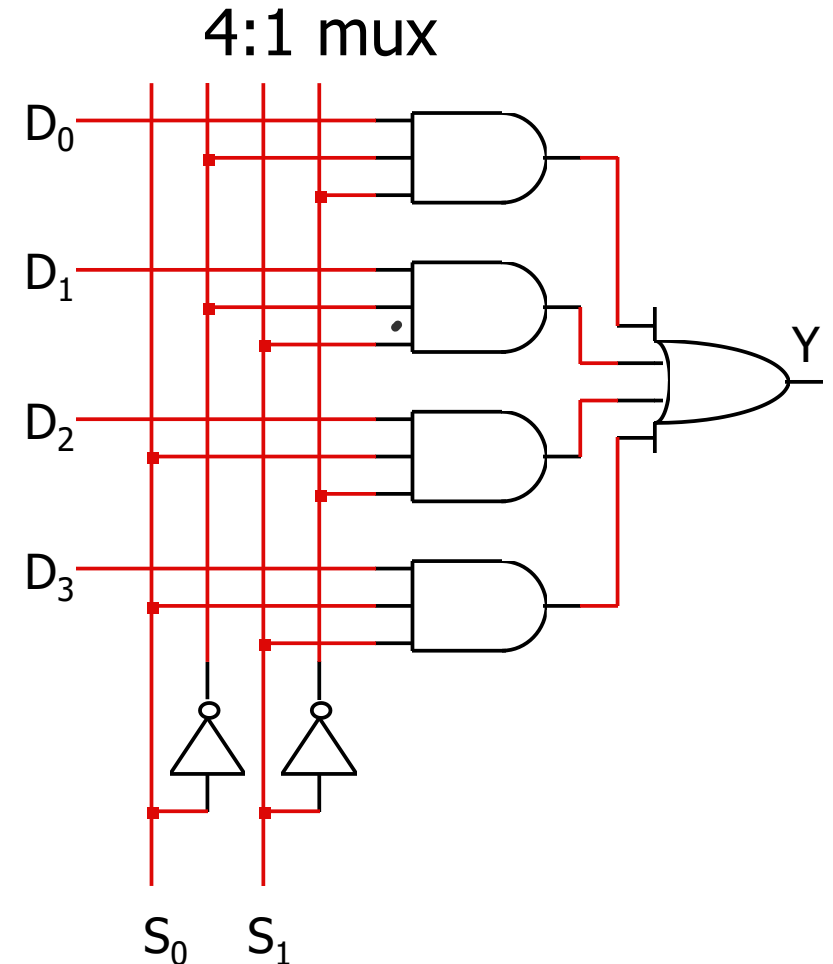
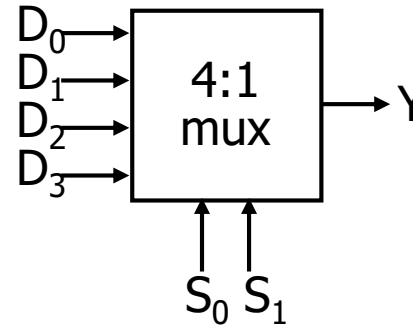
The output of the multiplexer is determined by the binary value of the selection lines

- When $S_1S_0=00$, the input D_0 is selected.
- When $S_1S_0=01$, the input D_1 is selected.
- When $S_1S_0=10$, the input D_2 is selected.
- When $S_1S_0=11$, the input D_3 is selected.

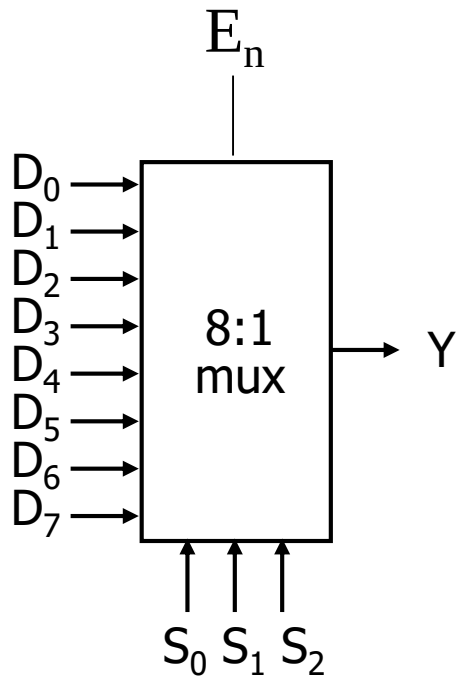
S_1	S_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

Functional truth table

$$Y = S_1'S_0'D_0 + S_1'S_0D_1 + S_1S_0'D_2 + S_1S_0D_3$$



8x1 Multiplexer



In the 8 to 1 multiplexer, there are total eight inputs, 3 selection lines, i.e., S_0 , S_1 and S_2 and single output, i.e., Y . On the basis of the combination of inputs that are present at the selection lines S_0 , S_1 , and S_2 , one of these 8 inputs are connected to the output. The block diagram and the truth table of the 8x1 multiplexer are given below.

Functional truth table

S_2	S_1	S_0	Y
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3
1	0	0	D_4
1	0	1	D_5
1	1	0	D_6
1	1	1	D_7

The Boolean expression for an 8-to-1-line multiplexer is:

$$f = (D_0 \bar{S}_2 \bar{S}_1 \bar{S}_0 + D_1 \bar{S}_2 \bar{S}_1 S_0 + D_2 \bar{S}_2 S_1 \bar{S}_0 + D_3 \bar{S}_2 S_1 S_0 + D_4 S_2 \bar{S}_1 \bar{S}_0 + D_5 S_2 \bar{S}_1 S_0 + D_6 S_2 S_1 \bar{S}_0 + D_7 S_2 S_1 S_0).$$

MUX Tree

Mux tree is used to obtain higher order MUX using Lower order MUX

to make a $p:1$ multiplexer using only a $q:1$ multiplexer. The general methodology to tackle such problems is to divide n by m until we get 1 and find the number of stages and the number of multiplexers required.

- - $p/q = k_1$ Stage 1
 - $k_1/q = k_2$ Stage 2
 - ∴
 - $k_2/q = k_3$ Stage 3 ∴ ∴
 - $k_{n-1}/q = k_n = 1$ Stage n

$$\text{total number of multiplexers} = k_1 + k_2 + \dots + k_{n-1} + 1$$

MUX Tree

How many 4×1 MUX required to get 8×1 MUX?

How many 2×1 MUX required to get 32×1 MUX?

How many 4×1 MUX required to get 16×1 MUX?

How many 2×1 MUX required to get 16×1 MUX?

4x1 MUX using 2x1 MUX

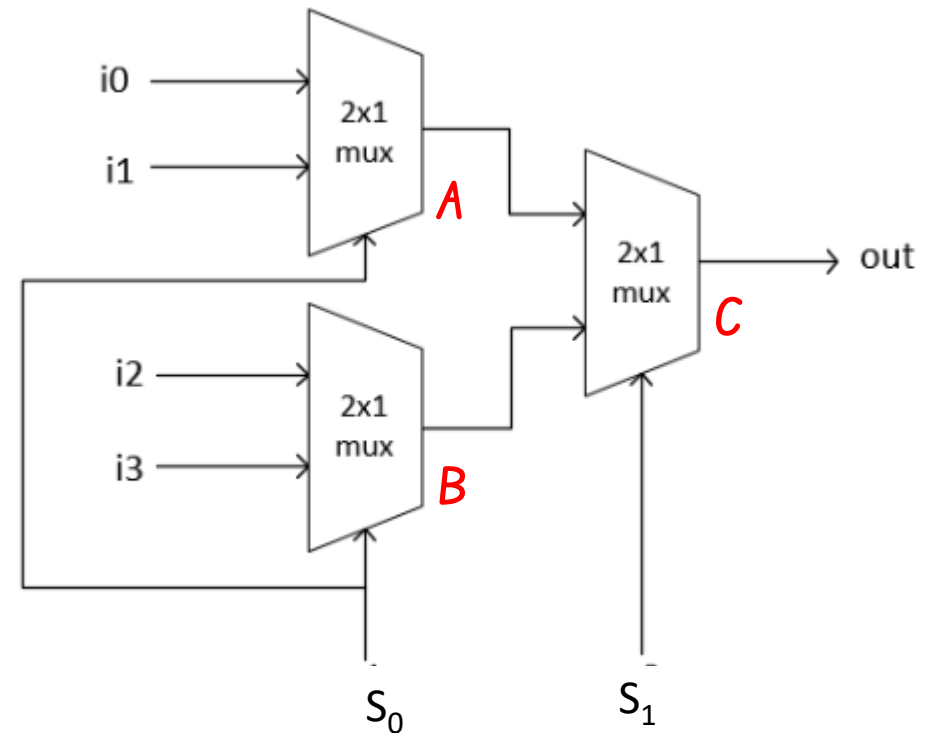
Working Principle:

- MUX A selects between I_0 and I_1 using S_0
- MUX B selects between I_2 and I_3 using S_0
- MUX C selects between the outputs of MUX A and MUX B using S_1

$$Y = S_1 \cdot (S_0 \cdot I_0 + S_0 \cdot I_1) + S_1 \cdot (S_0 \cdot I_2 + S_0 \cdot I_3)$$

Functional truth table

S_1	S_0	Y	
0	0	I_0	
0	1	I_1	
1	0	I_2	
1	1	I_3	



8x1 MUX using 2x1 MUX

Working Principle:

1.Stage 1:

The first stage of 2-to-1 MUXes reduces the 8 inputs into 4 intermediate values. These MUXes use S_0 as the select line to choose between pairs of inputs.

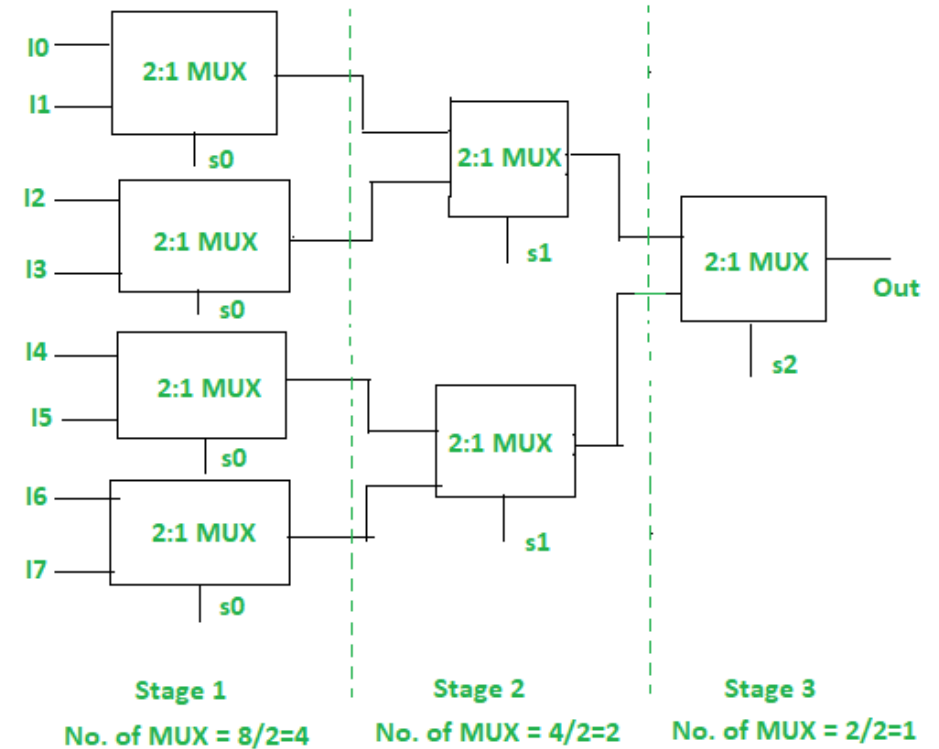
2.Stage 2:

The second stage reduces the 4 intermediate values into 2. These MUXes use S_1 as the select line to choose between pairs of values.

3.Stage 3:

The final MUX uses S_2 as the select line to choose between the two remaining values, thus producing the final output.

S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7



8x1 MUX using 4x1 MUX

An 8-to-1 Multiplexer (MUX) can also be constructed by using two 4-to-1 multiplexers and one additional 2-to-1 MUX.

Components:

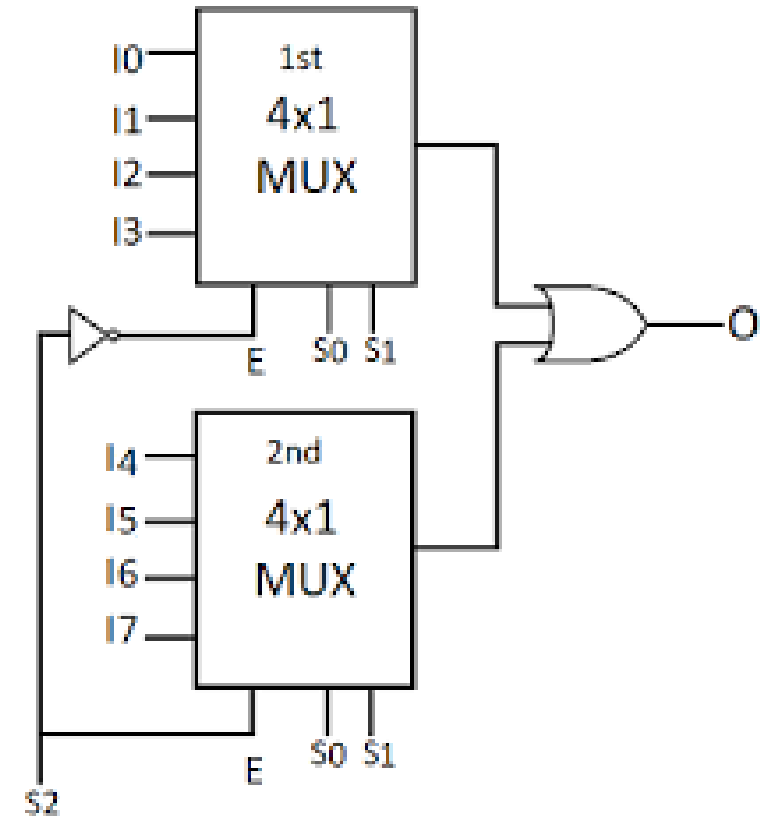
- Two 4x1 MUXs:

- 1st MUX: Inputs are I_0, I_1, I_2, I_3
- 2nd MUX: Inputs are I_4, I_5, I_6, I_7
- Both MUXs share the same selection lines S_0 and S_1

- Enable signals for MUXs:

- The 1st MUX is enabled when S_2' (NOT S_2) is HIGH (active low enable).
- The 2nd MUX is enabled when S_2 is HIGH.

- OR Gate: Takes outputs from both MUXs and gives final output O .



8x1 MUX using 4x1 MUX

Working:

The **selection lines** (S_0 and S_1) choose one of the 4 inputs in each MUX.

The S_2 signal determines **which MUX is active**:

If $S_2 = 0$:

$S_2' = 1 \rightarrow$ 1st MUX is enabled

2nd MUX is disabled

If $S_2 = 1$:

$S_2' = 0 \rightarrow$ 1st MUX is disabled

2nd MUX is enabled

So, effectively, this acts as an **8x1 multiplexer** built using **two 4x1 MUXs**.

S_2	S_1	S_0	Y	
0	0	0	I_0	1 st Mux is enabled
0	0	1	I_1	
0	1	0	I_2	
0	1	1	I_3	
1	0	0	I_4	2 nd Mux is enabled
1	0	1	I_5	
1	1	0	I_6	
1	1	1	I_7	

Functional truth table

MUX Example

□ Example-1: Implement the following function with a multiplexer: $F(A,B,C) = \Sigma(1,3,5,6)$

- ❖ If a Boolean function consists n variables,
 - Take n of these variables and connect them to control/selection lines.
 - Remaining single variable is used for inputs.
 - Either MSB or LSB is choose for inputs
 - Choosing MSB is easier than LSB

Implement Boolean function $F(A,B,C) = \sum(1,3,5,6)$

Truth Table

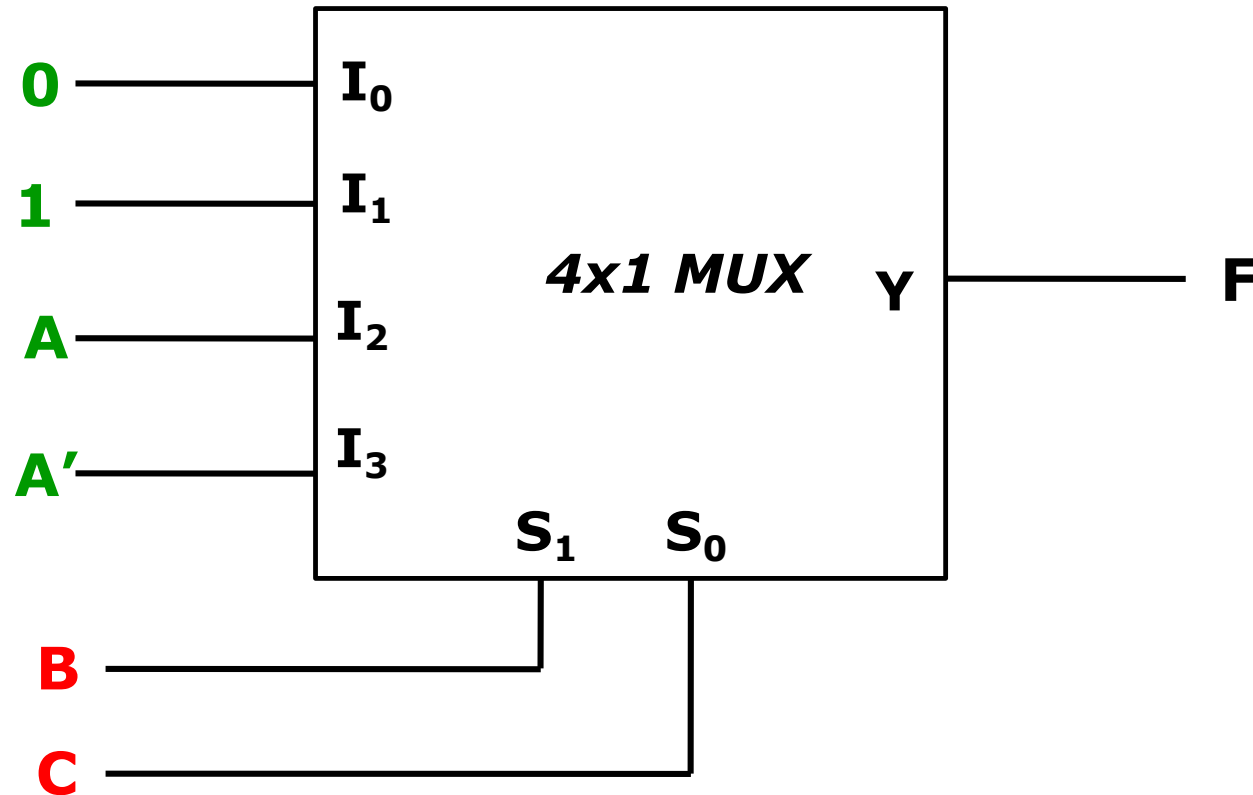
minterm	A	B	C	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

Implementation Table

	I ₀	I ₁	I ₂	I ₃
A'	0	1	2	3
A	4	5	6	7
	0	1	A	A'

- Choosing, B and C for selection lines and A (e.g., MSB) as inputs.
- ❖ 1st row listed minterms where A complemented.
- ❑ 2nd row listed minterms where A uncomplemented.
- ❖ If two minterms in a column
 - Not circled, apply 0 as MUX input.
 - Circled, apply 1 as MUX input.
- ❖ If bottom minterm circled and top not, apply A as MUX input.
- ❖ For opposite case, apply A'

Implement Boolean function $F(A,B,C) = \sum(1,3,5,6)$



Implementation Table

	I_0	I_1	I_2	I_3
A'	0	1	2	3
A	4	5	6	7
	0	1	A	A'

Multiplexer implementation

Implement Boolean function $F(A,B,C) = \Sigma(1,3,5,6)$

➤ Choosing, LSB as inputs

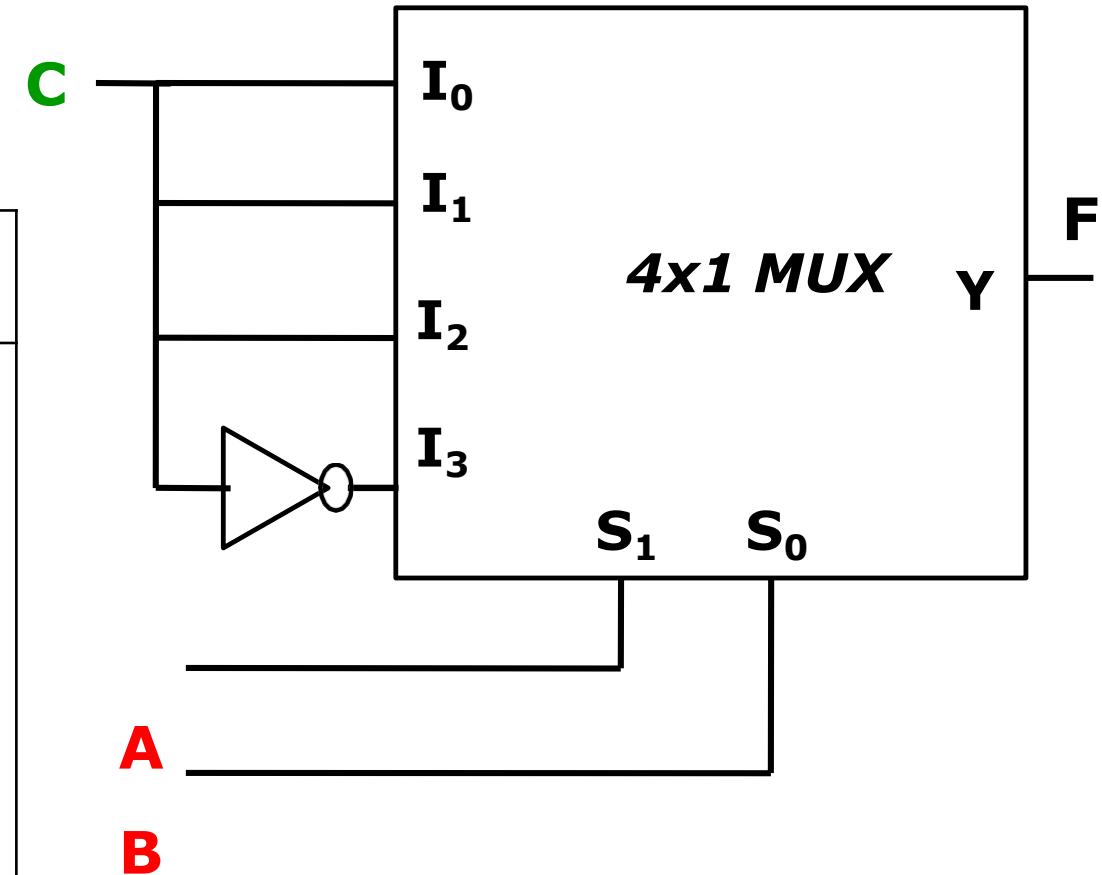
$$F(A,B,C) = \Sigma(1,3,5,6)$$

Implementation Table

	I_0	I_1	I_2	I_3
C'	0	2	4	6
C	1	3	5	7
	C	C	C	C'

Truth Table

	minterm	A	B	C	F
0	0	0	0	0	0
1	1	0	0	1	1
2	2	0	1	0	0
3	3	0	1	1	1
4	4	1	0	0	0
5	5	1	0	1	1
6	6	1	1	0	1
7	7	1	1	1	0

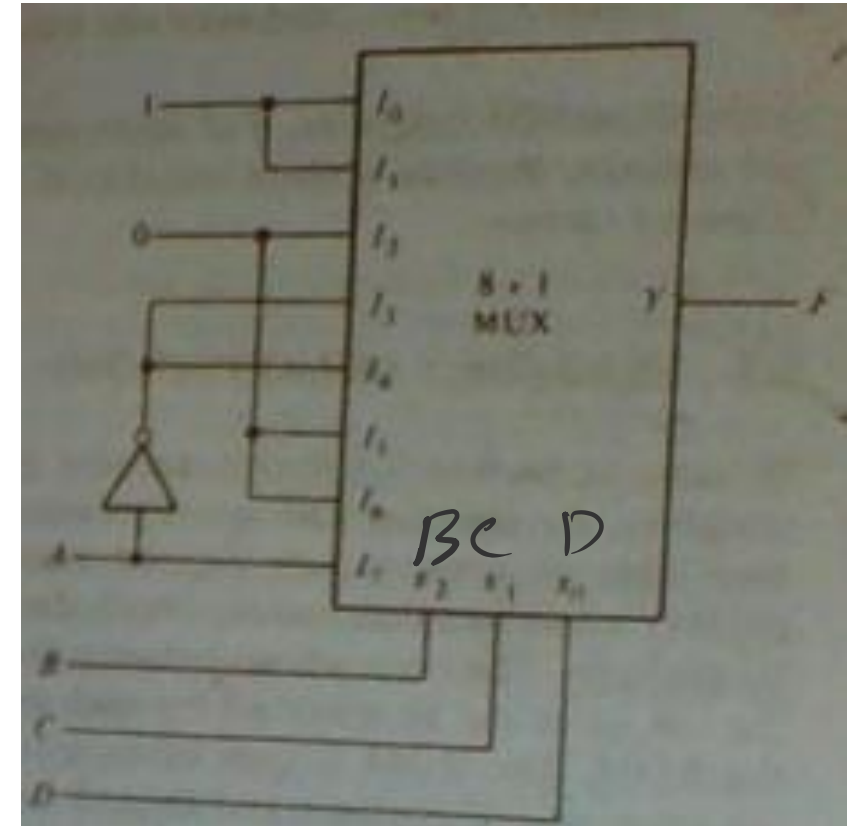


Implement Boolean function with a multiplexer

$$F(A, B, C, D) = \sum(0, 1, 3, 4, 8, 9, 15)$$

➤ Choosing, MSB or A as inputs

	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
A'	0	1	2	3	4	5	6	7
A	8	9	10	11	12	13	14	15
	1	1	0	A'	A'	0	0	A



Implement Full adder with a 4x1 multiplexer

x	y	z	c	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1 ✓

$$S(x,y,z) = \Sigma(1,2,4,7)$$

$$C(x,y,z) = \Sigma(3,5,6,7)$$

For S

	I ₀	I ₁	I ₂	I ₃
x'	0	1	2	3
x	4	5	6	7
	x	x'	x'	x

For C

	I ₀	I ₁	I ₂	I ₃
x'	0	1	2	3
x	4	5	6	7
	0	x	x	1

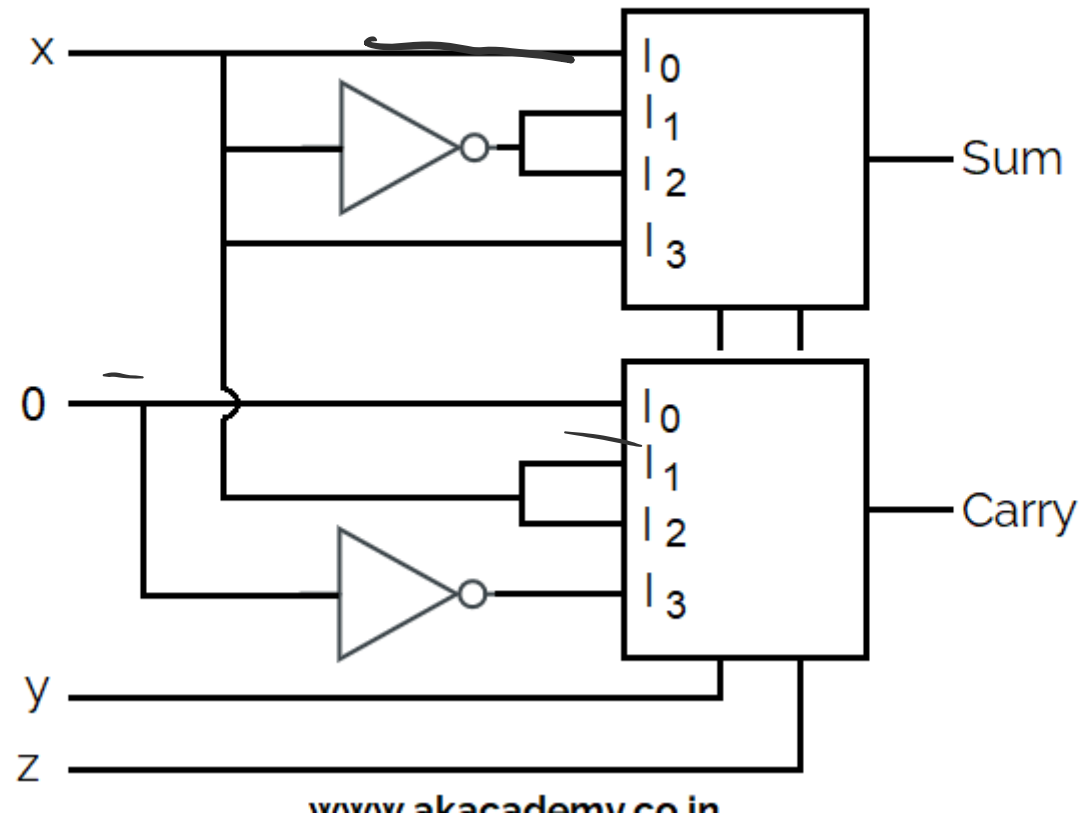
Implement Full adder with a 4x1 multiplexer

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$S(x,y,z)=\Sigma(1,2,4,7)$$

$$C(x,y,z)=\Sigma(3,5,6,7)$$

Logic Diagram of FULL Adder Using 4X1 Multiplexers



Implement full adder with a 2x1 multiplexer

	x	y	z	C	S
0	0	0	0	0	0
1	0	0	1	0	1 ✓
2	0	1	0	0	1 ✓
3	0	1	1	1 ✓	0
4	1	0	0	0	1 ✓
5	1	0	1	1	0
6	1	1	0	1	0
7	1	1	1	1	1

$$S(x,y,z) = \Sigma(1,2,4,7)$$

$$C(x,y,z) = \Sigma(3,5,6,7)$$

For S

	I0	I1
X'Y'	0	1
X'Y	2	3
XY'	4	5
XY	6	7

For S

$$I_0 = X'Y + XY'$$

$$I_1 = X'Y' + XY = (X'Y + XY')'$$

For C

	I0	I1
X'Y'	0	1
X'Y	2	3
XY'	4	5
XY	6	7

For C

$$I_0 = XY$$

$$I_1 = X'Y + XY' + XY$$

$$= X'Y + X(Y' + Y)$$

$$= X'Y + X$$

$$= (X' + X)(X + Y)$$

$$= X + Y$$

Implement full adder with a 2x1 multiplexer

	x	y	z	C	S
0	0	0	0	0	0
1	0	0	1	0	1
2	0	1	0	0	1
3	0	1	1	1	0
4	1	0	0	0	1
5	1	0	1	1	0
6	1	1	0	1	0
7	1	1	1	1	1

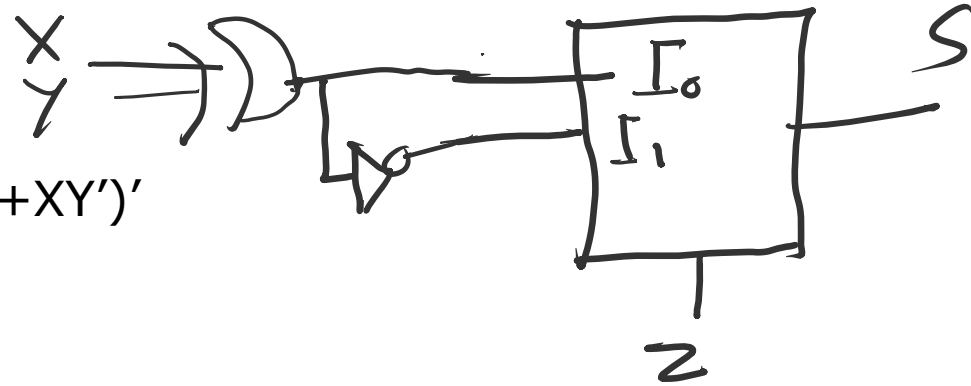
$$S(x,y,z) = \Sigma(1,2,4,7)$$

$$C(x,y,z) = \Sigma(3,5,6,7)$$

For S

$$I_0 = X'Y + XY'$$

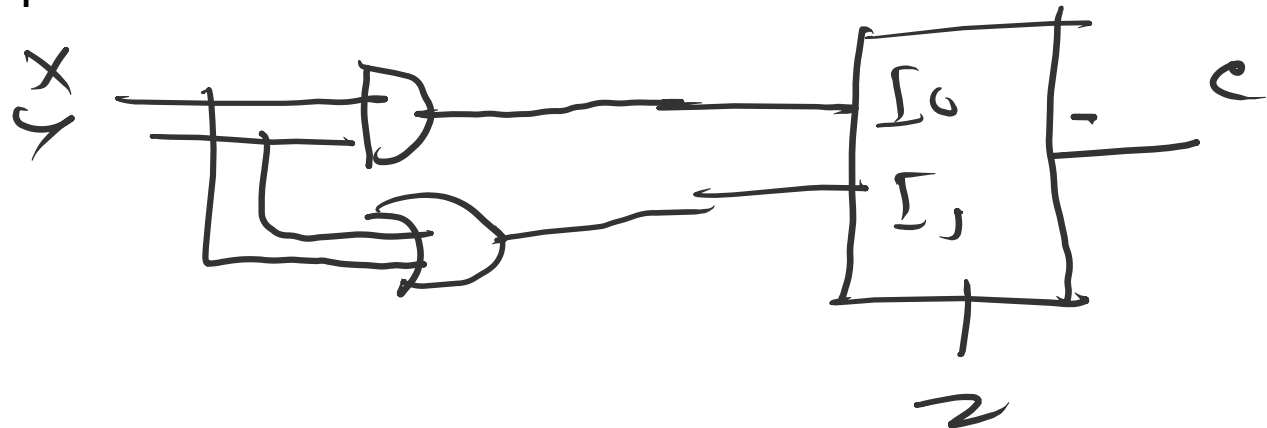
$$I_1 = X'Y' + XY = (X'Y + XY')'$$



For C

$$I_0 = XY$$

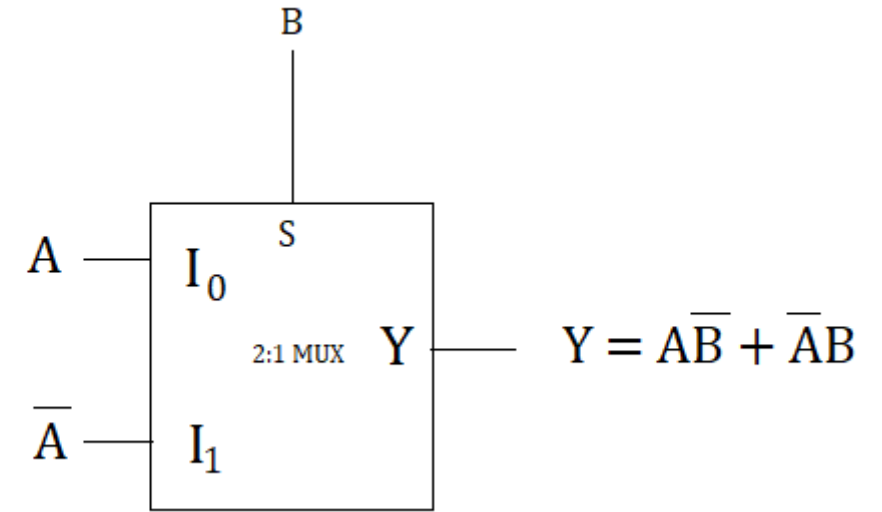
$$I_1 = X + Y$$



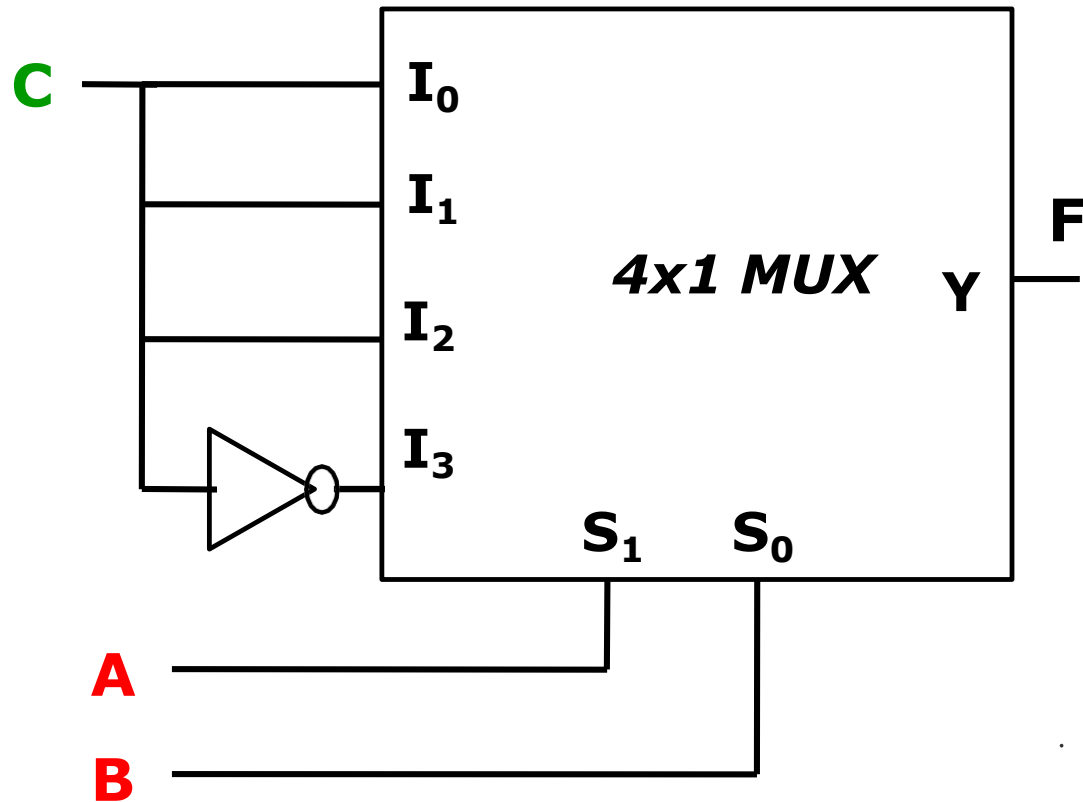
Implement X-OR gate with a 2x1 multiplexer

	A	B	Y
0	0	0	0
1	0	1	1
2	<u>1</u>	0	1
3	1 ←	1	0

	I ₀	I ₁
A'	0	1
A	2	3
	A ←	A' ✓



Derive the expression of the function



Multiplexer implementation

$$Y = S_0'S_1'I_0 + S_0'S_1I_1 + S_0S_1'I_2 + S_0SI_3$$

$$Y = A'B'C + A'BC + AB'C + ABC'$$

$$Y = A'C(B' + B) + AB'C + ABC'$$

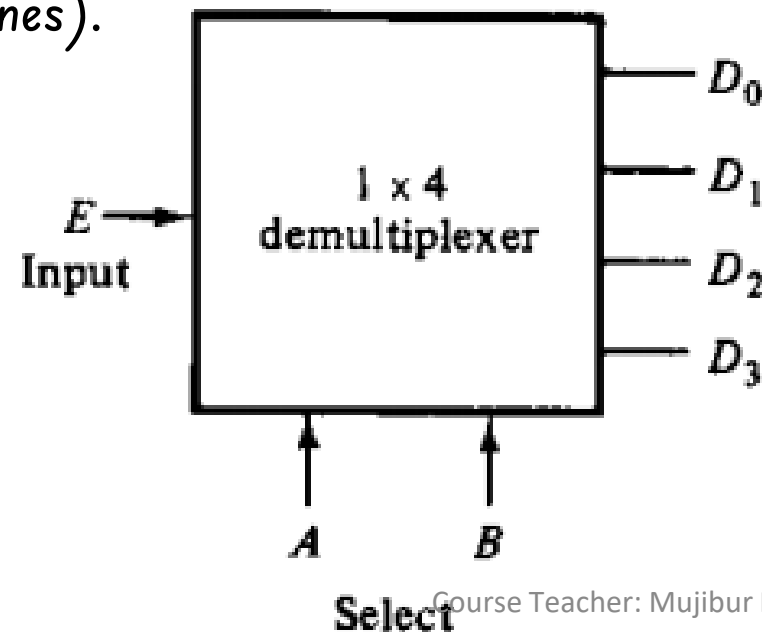
$$Y = C(A' + AB') + ABC'$$

$$Y = C(A' + B') + ABC'$$

$$= CA' + CB' + ABC'$$

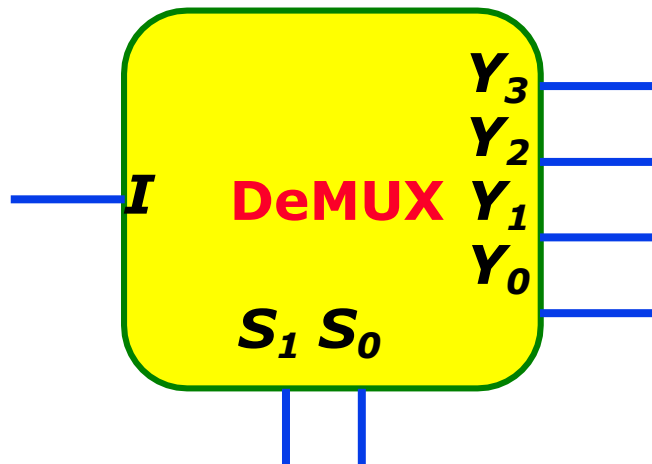
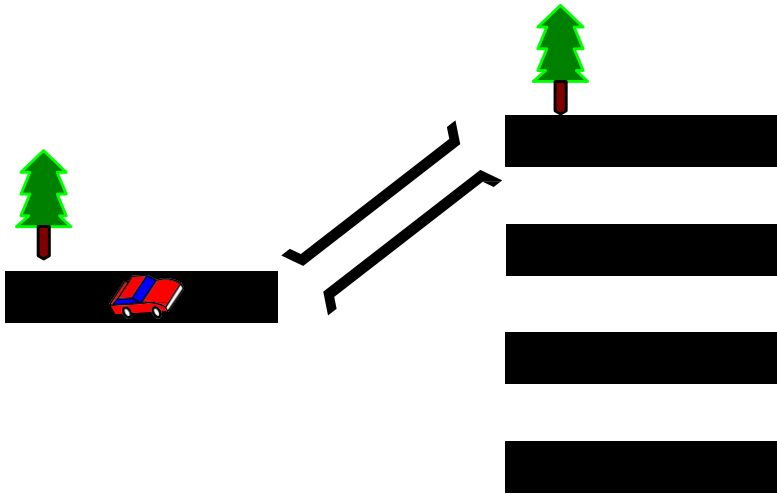
DeMultiplexers

- ❑ A circuit that receives information on a single line and transmit this information on one of 2^n possible output lines.
- Selection of a specific output line is controlled by bit values of n selection lines.
- To be able to select among n inputs, $\log_2 n$ control lines are needed (2^n input lines and n selection lines).



Block diagram of a Demultiplexer

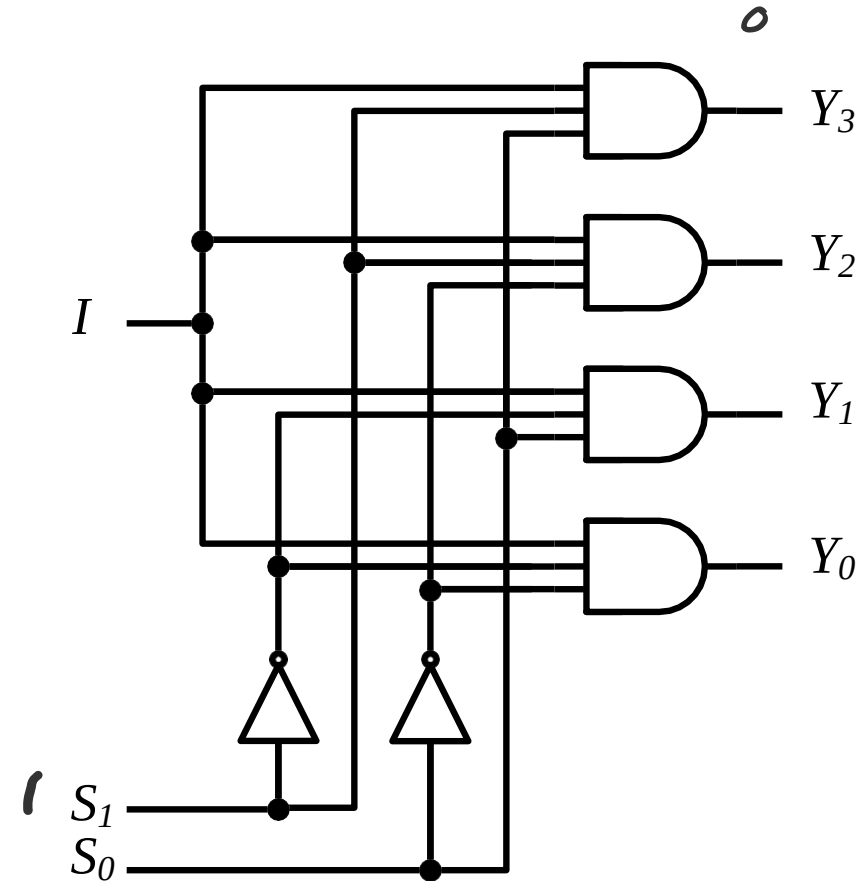
1x4 DeMultiplexer



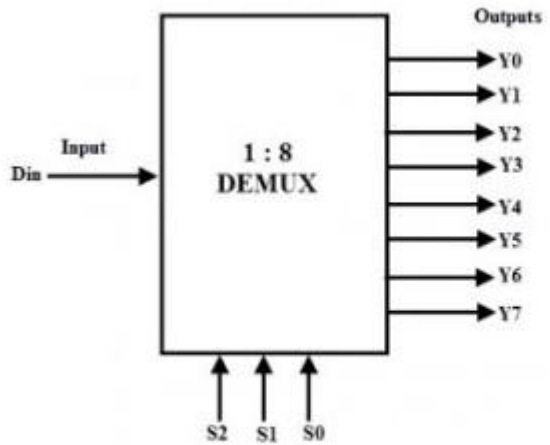
S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

$$Y_3 = I \cdot S_1 S_0 \quad Y_2 = I \cdot S_1 S_0'$$

$$Y_1 = I \cdot S_1' S_0 \quad Y_0 = I \cdot S_1' S_0'$$



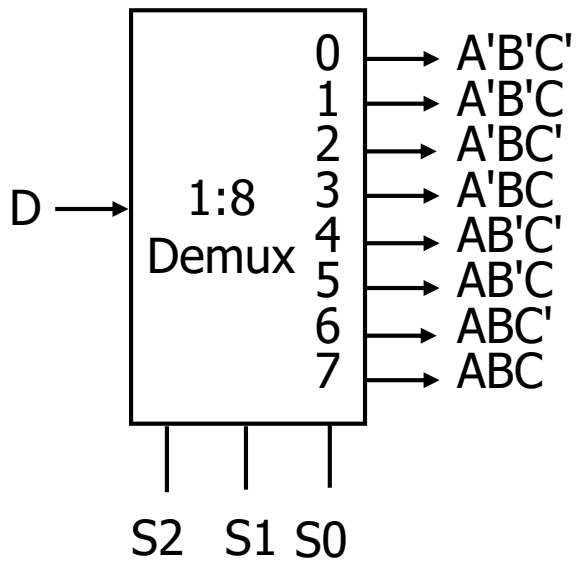
1x8 DeMultiplexer



1 to 8 Demux Block Diagram

Data Input	Select Inputs			Outputs							
D	S ₂	S ₁	S ₀	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀
D	0	0	0	0	0	0	0	0	0	0	D
D	0	0	1	0	0	0	0	0	0	D	0
D	0	1	0	0	0	0	0	0	D	0	0
D	0	1	1	0	0	0	0	D	0	0	0
D	1	0	0	0	0	0	D	0	0	0	0
D	1	0	1	0	0	D	0	0	0	0	0
D	1	1	0	0	D	0	0	0	0	0	0
D	1	1	1	D	0	0	0	0	0	0	0

1 to 8 Demux Truth Table



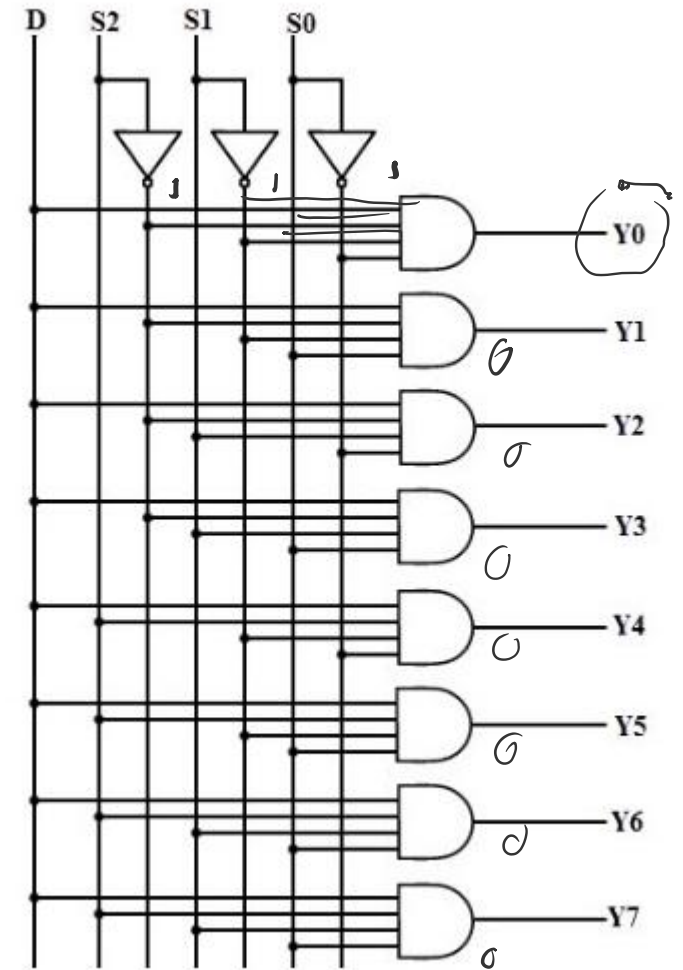
2-to-4 Decoder can be used as 1-to-4 DMUX

S₂=A

S₁=B

S₀=C

Course Teacher: Mujibur Rahman Maruf



Demultiplexers as general-purpose logic

Example

$$F1 = A'BC'D + A'B'CD + ABCD$$

$$F1 = \sum(3, 5, 15)$$

$$F2 = ABC'D' + ABC$$

$$F2 = \sum(12, 14, 15)$$

$$F3 = (A' + B' + C' + D')$$

$$F3 = (ABCD)'$$

